

## Tabla de contenido

|                                                                     |    |
|---------------------------------------------------------------------|----|
| 1. Introducción: de la IA clásica al deep learning. ....            | 2  |
| 2. La neurona artificial y el perceptrón. ....                      | 2  |
| 3. Del perceptrón simple al perceptrón multicapa. ....              | 7  |
| 4. Funciones de activación: por qué son necesarias. ....            | 11 |
| 5. Entrenamiento de redes neuronales. ....                          | 13 |
| 6. Problemas frecuentes en el entrenamiento. ....                   | 16 |
| 7. Tipos de redes neuronales y campos de aplicación. ....           | 17 |
| 8. Comparación con otros métodos de inteligencia artificial. ....   | 19 |
| 9. Cómo elegir el modelo adecuado según el problema. ....           | 21 |
| 10. Cómo reconocer una red entrenada y valorar su rendimiento. .... | 22 |
| 11. Ejemplos guiados paso a paso. ....                              | 25 |
| 12. Ampliación. ....                                                | 26 |

# 1. Introducción: de la IA clásica al deep learning.

## 1.1. Qué lugar ocupan las redes neuronales dentro de la inteligencia artificial.

Cuando hablamos de inteligencia artificial solemos mezclar varias capas de significado. En sentido amplio, la IA es el campo que diseña sistemas capaces de realizar tareas que normalmente asociamos con la inteligencia humana: clasificar, tomar decisiones, reconocer patrones, predecir comportamientos o generar contenido. Dentro de esa IA encontramos el aprendizaje automático o machine learning, es decir, los métodos que aprenden a partir de datos en vez de depender únicamente de reglas escritas a mano.

Las **redes neuronales artificiales** pertenecen al machine learning. Son una familia de modelos que **aprenden ajustando pesos y sesgos para transformar entradas en salidas útiles**. Cuando estas redes tienen varias **capas ocultas** y se entrenan sobre grandes volúmenes de datos, hablamos de **aprendizaje profundo o deep learning**. Por tanto, deep learning no es un campo independiente, sino una especialización del aprendizaje automático basada en redes neuronales profundas.

Esta distinción es importante para el alumnado, porque no hay que saber qué es una red neuronal, sino entender cuándo conviene usarla y cuándo otro método de IA puede ser más adecuado. Esa capacidad de comparar y elegir es uno de los indicadores más claros de madurez técnica.

**Idea clave.** No toda IA usa redes neuronales y no todo problema mejora por usar deep learning. La herramienta correcta depende del tipo de dato, del volumen de ejemplos disponibles, del coste computacional y del nivel de explicabilidad que exija el contexto.

## 1.2. Por qué las redes neuronales han tenido tanto éxito.

Las redes neuronales existen desde hace décadas, pero su expansión real se ha producido en los últimos años por tres factores que han coincidido en el tiempo. Primero, hoy disponemos de muchos más datos digitales que antes: imágenes, texto, registros de sensores, audio o historiales de uso. Segundo, el hardware actual permite entrenar modelos con millones de parámetros en un tiempo razonable. Tercero, los algoritmos de optimización y las arquitecturas han mejorado mucho respecto a las primeras propuestas.

En visión artificial, lenguaje natural, reconocimiento de voz o recomendación, las redes profundas han demostrado una capacidad extraordinaria para aprender patrones complejos. Sin embargo, esa potencia tiene un coste: suelen requerir más datos, más tiempo de entrenamiento y más cuidado en la validación. Un árbol de decisión o una regresión logística pueden resolver mejor que una red neuronal un problema pequeño, con pocos datos y necesidad alta de interpretación.

# 2. La neurona artificial y el perceptrón.

## 2.1. La neurona artificial como unidad básica de cálculo.

Una neurona artificial recibe varias entradas numéricas, las combina mediante pesos y añade un sesgo o bias. El resultado de esa combinación se suele denotar por  $z$ . A continuación, una función de activación transforma  $z$  en una salida  $a$ . En su forma más simple podemos escribir la neurona así:  $z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$ .

Los pesos indican la importancia de cada variable de entrada. Si un peso es grande y positivo, esa entrada empuja la salida hacia arriba; si es negativo, la frena; si es cercano a cero, su impacto es pequeño. El sesgo

permite desplazar el umbral de decisión aunque todas las entradas valgan cero. Esta idea es análoga a la ordenada en el origen de una recta.

**Figura 1. Del perceptrón al perceptrón multicapa (MLP)**

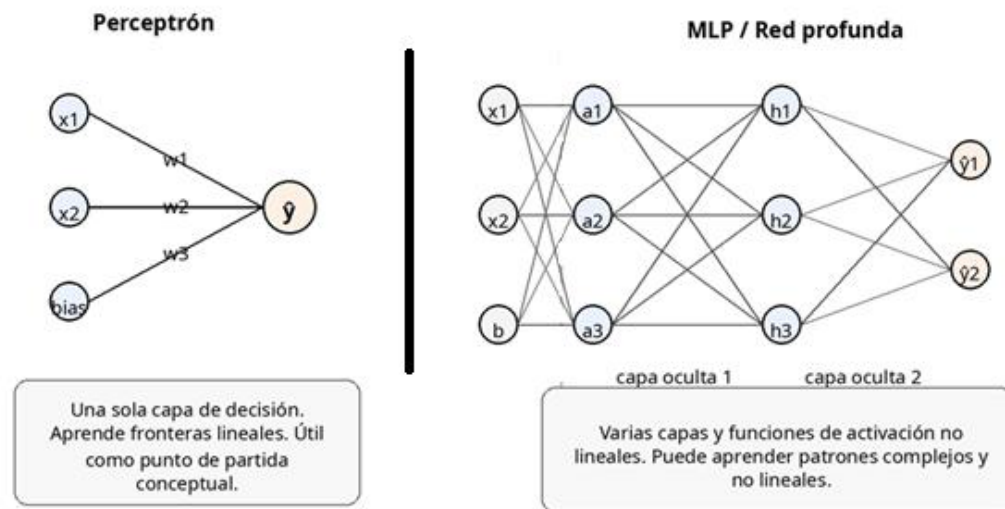


Figura 1. Comparación visual entre un perceptrón simple y un perceptrón multicapa.

## 2.2. Qué hace realmente una TLU o unidad lógica de umbral.

El perceptrón histórico se apoya en una neurona muy sencilla llamada TLU, del inglés Threshold Logic Unit. La TLU calcula una combinación lineal de las entradas y aplica después una función escalón. Si el valor supera un cierto umbral, la salida vale 1; si no lo supera, la salida vale 0. Esto convierte a la neurona en un clasificador binario elemental.

Desde el punto de vista didáctico, la TLU es útil porque permite conectar ideas ya conocidas por el alumnado, como la suma ponderada de variables, con la clasificación. De hecho, su estructura se parece mucho a modelos lineales estudiados en otras unidades, aunque su salida suele ser discreta en lugar de continua.

## 2.3. El perceptrón: primera red neuronal funcional.

El perceptrón es una red formada por una capa de entrada y una capa de salida. Cada neurona de salida recibe todas las entradas y produce una clasificación. En problemas muy sencillos puede comportarse como un filtro capaz de decidir entre dos clases, por ejemplo 'spam/no spam', 'aprobado/suspenso' o 'incidencia/no incidencia'.

Si solo usamos una neurona de salida, el perceptrón realiza una clasificación binaria. Si usamos varias neuronas de salida, cada una puede asociarse a una clase, aunque en las implementaciones modernas solemos recurrir a funciones y arquitecturas más ricas que las originales. Aun así, el perceptrón sigue siendo el mejor punto de partida para entender cómo se construye una red neuronal.

## 2.4. Ejemplo resuelto: puerta lógica AND con un perceptrón.

Supongamos que queremos que la salida valga 1 únicamente cuando las dos entradas son 1. La tabla de verdad de la puerta AND es: (0,0)→0, (0,1)→0, (1,0)→0 y (1,1)→1. Esto puede implementarse con un perceptrón si elegimos, por ejemplo, pesos  $w_1 = 1$ ,  $w_2 = 1$  y sesgo  $b = -1.5$ . Entonces  $z = x_1 + x_2 - 1.5$ .

Al evaluar cada caso, obtenemos  $z = -1.5, -0.5, -0.5$  y  $0.5$ . Si usamos una función escalón que devuelve 1 cuando  $z$  es positivo y 0 en otro caso, la salida coincide exactamente con la tabla AND. Este ejemplo demuestra que el perceptrón puede aprender relaciones linealmente separables.

El interés del ejemplo no está solo en la solución, sino en la intuición que genera: un perceptrón define una frontera lineal de decisión. En dos variables, esa frontera es una recta; en tres, un plano; en muchas dimensiones, un hiperplano.

| x1 | x2 | $z = 1 \cdot x_1 + 1 \cdot x_2 - 1.5$ | salida |
|----|----|---------------------------------------|--------|
| 0  | 0  | -1.5                                  | 0      |
| 0  | 1  | -0.5                                  | 0      |
| 1  | 0  | -0.5                                  | 0      |
| 1  | 1  | 0.5                                   | 1      |

## 2.5. Cómo se entrena un perceptrón.

### 2.5.1. Qué significa optimizar un perceptrón.

Cuando hablamos de optimización en aprendizaje automático nos referimos al proceso de encontrar los valores más adecuados de los parámetros del modelo. En el caso del perceptrón, esos parámetros son los pesos y el sesgo o bias, es decir, los valores que determinan cómo influyen las variables de entrada en la salida final. Si una neurona recibe dos entradas  $x_1$  y  $x_2$ , sus parámetros serán  $w_1$ ,  $w_2$  y  $b$ . Si trabajamos con varias TLUs en la capa de salida, cada una tendrá sus propios parámetros. Por ejemplo, con tres TLUs tendríamos conjuntos como  $(w_{11}, w_{21}, b_1)$ ,  $(w_{12}, w_{22}, b_2)$  y  $(w_{13}, w_{23}, b_3)$ .

La idea general es la misma que en otros algoritmos vistos anteriormente: comenzar con unos parámetros iniciales y modificarlos poco a poco hasta que el modelo cometa el menor número posible de errores. Sin embargo, en el perceptrón clásico este ajuste es más sencillo que en modelos como la regresión logística. En regresión logística se parte de una función de error basada en logaritmos y se emplea descenso por gradiente para minimizarla; en cambio, en el perceptrón la corrección de los pesos se hace mediante una regla directa basada en la diferencia entre la etiqueta real y la predicción obtenida.

### 2.5.2. Regla de actualización de los pesos.

El perceptrón calcula primero una combinación lineal de las entradas:

$$z = w_1x_1 + w_2x_2 + b$$

A continuación, aplica una función de activación, normalmente una función escalón como Heaviside, para obtener la salida predicha. Esa salida suele representarse como  $\hat{y}$ . Si la predicción no coincide con la etiqueta real del ejemplo de entrenamiento, entonces se modifican los parámetros.

La regla general de actualización del perceptrón puede escribirse así:

$$w_{ij}(\text{nuevo}) = w_{ij}(\text{viejo}) + \eta (y_j - \hat{y}_j) x_i$$

$$b_j(\text{nuevo}) = b_j(\text{viejo}) + \eta (y_j - \hat{y}_j)$$

En esta expresión,  $\eta$  es la tasa de aprendizaje o learning rate,  $y_j$  es la etiqueta real del ejemplo,  $\hat{y}_j$  es la predicción realizada por la TLU y  $x_i$  es el valor de la variable de entrada correspondiente. Esta regla indica algo muy importante: los pesos solo cambian cuando el modelo comete un error. Si la predicción ya es correcta, la diferencia  $(y_j - \hat{y}_j)$  vale 0 y no se produce ninguna modificación.

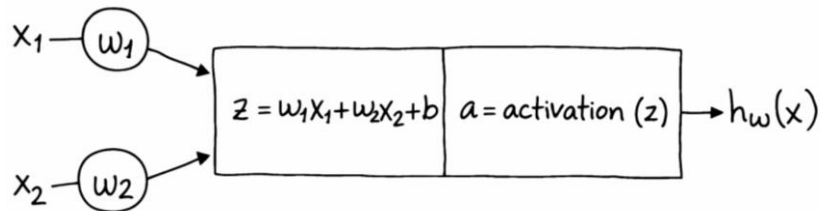
### 2.5.3. Papel del learning rate.

El learning rate o tasa de aprendizaje es un hiperparámetro que controla el tamaño del cambio que aplicamos a los pesos en cada actualización. Es decir, determina cuánto corregimos el modelo cuando detectamos un error. Esta idea también aparece en la explicación general del descenso por gradiente: si los pasos son

demasiado grandes, el entrenamiento puede volverse inestable; si son demasiado pequeños, el aprendizaje será muy lento.

En el perceptrón ocurre algo similar. Si  $\eta$  es muy grande, los pesos pueden variar demasiado y el entrenamiento puede oscilar. Si  $\eta$  es muy pequeña, el aprendizaje será más estable, pero también más lento. Por tanto, el learning rate actúa como un regulador de la velocidad de aprendizaje del modelo.

#### 2.5.4. Entrenamiento paso a paso de una TLU.



Para comprender mejor la optimización del perceptrón, vamos a seguir el entrenamiento de una sola TLU. Supongamos que tenemos dos características de entrada,  $x_1$  y  $x_2$ , y una etiqueta real  $y$ . La neurona calcula:

$$z = w_{11}x_1 + w_{21}x_2 + b_1$$

Después aplica la función de activación:

$$\hat{y}_1 = \text{Heaviside}(z)$$

Al principio, los valores  $w_{11}$ ,  $w_{21}$  y  $b_1$  se inicializan de manera arbitraria o aleatoria. Como estos valores todavía no son buenos, es normal que las primeras predicciones sean erróneas. Precisamente por eso es necesario entrenar el modelo.

El procedimiento completo es el siguiente:

1. Se toma un ejemplo del conjunto de entrenamiento.
2. Se calculan la combinación lineal  $z$  y la predicción  $\hat{y}$ .
3. Se compara la predicción con la etiqueta real  $y$ .
4. Si hay error, se actualizan pesos y sesgo.
5. Se repite el proceso con muchos ejemplos hasta mejorar el rendimiento del modelo.

#### 2.5.5. Qué ocurre si la predicción es correcta.

Supongamos que para un ejemplo concreto la etiqueta real vale 1 y la neurona también predice 1. En ese caso:

$$(y - \hat{y}) = (1 - 1) = 0$$

Entonces la regla de actualización queda:

$$w_{11}(\text{nuevo}) = w_{11}(\text{viejo})$$

$$w_{21}(\text{nuevo}) = w_{21}(\text{viejo})$$

$$b_1(\text{nuevo}) = b_1(\text{viejo})$$

Es decir, no se modifica ningún parámetro, porque el perceptrón ya ha acertado y no necesita corregirse. Esta es una idea muy importante: el modelo solo cambia cuando detecta que su decisión ha sido incorrecta.

### 2.5.6. Qué ocurre si la predicción es errónea.

Ahora supongamos que la etiqueta real sigue siendo 1, pero la neurona predice 0. Entonces:

$$(y - \hat{y}) = (1 - 0) = 1$$

En este caso sí se actualizan los pesos:

$$w_{11}(\text{nuevo}) = w_{11}(\text{viejo}) + \eta \cdot x_1$$

$$w_{21}(\text{nuevo}) = w_{21}(\text{viejo}) + \eta \cdot x_2$$

$$b_1(\text{nuevo}) = b_1(\text{viejo}) + \eta$$

Lo que está haciendo el algoritmo es mover los parámetros en una dirección que favorezca que, en futuras iteraciones, la neurona sea más propensa a clasificar correctamente ese ejemplo como positivo.

También puede ocurrir el caso contrario: que la etiqueta real sea 0 y la neurona prediga 1. Entonces  $(y - \hat{y}) = (0 - 1) = -1$ . En esa situación, los pesos y el sesgo disminuyen, corrigiendo el modelo en sentido opuesto. Esto ayuda a entender que el perceptrón no solo “premia” respuestas correctas, sino que corrige activamente los errores en la dirección adecuada.

### 2.5.7. Ejemplo numérico sencillo.

Supongamos el siguiente ejemplo de entrenamiento:  $x_1 = 1$ ,  $x_2 = 0.5$ , etiqueta real  $y = 1$ , predicción actual  $\hat{y} = 0$  y tasa de aprendizaje  $\eta = 0.2$ . Tomamos además los pesos iniciales  $w_{11} = 0.1$ ,  $w_{21} = -0.3$  y  $b_1 = 0$ .

Aplicando la regla de actualización obtenemos:

$$w_{11}(\text{nuevo}) = 0.1 + 0.2(1 - 0)(1) = 0.3$$

$$w_{21}(\text{nuevo}) = -0.3 + 0.2(1 - 0)(0.5) = -0.2$$

$$b_1(\text{nuevo}) = 0 + 0.2(1 - 0) = 0.2$$

Después de esta actualización, la neurona ha corregido sus parámetros para intentar clasificar mejor ese ejemplo en la siguiente iteración. Este tipo de cálculo es muy útil porque permite ver el aprendizaje del modelo de forma concreta y no como un proceso abstracto.

### 2.5.8. Relación con Gradient Descent.

Aunque el perceptrón clásico utiliza una regla de actualización más simple que otros modelos, la idea de fondo se parece a la del descenso por gradiente: modificar los parámetros para reducir el error. En redes neuronales más avanzadas, como el perceptrón multicapa, esta idea se generaliza mediante funciones de coste diferenciables, cálculo de gradientes y backpropagation. Por eso, estudiar la optimización del perceptrón resulta muy útil como paso previo para comprender cómo se entrenan redes neuronales más profundas.

El entrenamiento del perceptrón puede resumirse así: predecir → comparar con la realidad → corregir pesos → volver a predecir. Ese ciclo se repite muchas veces con ejemplos etiquetados hasta que el modelo mejora su capacidad de clasificación. La gran ventaja didáctica del perceptrón es que permite ver de forma clara cómo una red neuronal aprende ajustando parámetros a partir de sus errores. Esta lógica básica será la misma en redes más avanzadas, aunque allí el proceso matemático y computacional sea más complejo.

## 2.6. Limitaciones del perceptrón.

La principal limitación del perceptrón es que solo puede aprender fronteras lineales. Esto quiere decir que si las clases no pueden separarse mediante una recta o hiperplano, el modelo fracasará. El ejemplo clásico es

el problema XOR: cuando las salidas positivas aparecen en diagonal, ninguna frontera lineal separa correctamente las clases.

Además, el perceptrón original ofrece una salida muy rígida, poco adecuada para interpretar probabilidades. Tampoco escala bien a problemas ricos en estructura espacial o temporal, como imágenes, texto o series. Estas carencias explican por qué el perceptrón simple tiene hoy un valor principalmente conceptual, mientras que las aplicaciones reales se apoyan en redes multicapa y técnicas de deep learning.

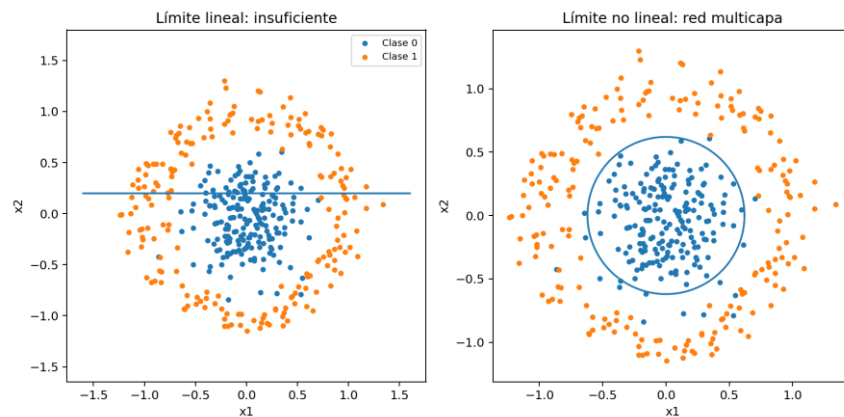


Figura 2. Un perceptrón simple solo puede construir límites de decisión lineales; un modelo multicapa puede aprender fronteras no lineales.

**Error frecuente del alumnado.** Pensar que una red neuronal es siempre 'mejor' que un modelo lineal. En realidad, una red pequeña sin los datos adecuados puede rendir peor que una regresión logística o un árbol. La complejidad del modelo debe estar justificada por la complejidad del problema.

### 3. Del perceptrón simple al perceptrón multicapa.

#### 3.1. Por qué una sola capa no basta.

Muchos problemas reales no presentan una separación lineal clara. En un conjunto de datos tabulares puede haber interacciones entre variables; en una imagen, la información relevante depende de patrones espaciales; en texto, el significado depende del contexto y del orden. Una sola capa de decisión es demasiado pobre para modelar estos fenómenos.

El salto fundamental consiste en añadir capas ocultas. Cada capa transforma la representación anterior y permite construir combinaciones progresivamente más abstractas. La salida de una capa se convierte en entrada de la siguiente. Gracias a esa composición de transformaciones, la red puede aproximar funciones mucho más complejas que una recta o un hiperplano.

#### 3.2. Estructura de un perceptrón multicapa (MLP).

Un perceptrón multicapa o MLP incorpora, al menos, una capa oculta entre la entrada y la salida. Si existen dos o más capas ocultas, solemos hablar ya de red profunda. Cada neurona de una capa suele estar conectada con todas las neuronas de la siguiente; por eso al MLP también se le llama red fully connected o totalmente conectada.

En un MLP, cada neurona repite el mismo esquema básico: calcula una combinación lineal, añade sesgo y aplica una activación. Lo que cambia es que las entradas ya no son las variables originales, sino las salidas

generadas por la capa anterior. Este encadenamiento es lo que permite a la red construir representaciones internas útiles.

### 3.2.1. Notación general de una neurona en un MLP.

La operación básica que realiza cualquier neurona de una red multicapa puede expresarse en dos pasos. Primero se calcula una combinación lineal de las activaciones procedentes de la capa anterior y, después, se aplica una función de activación. Esta notación es útil porque permite describir de manera uniforme cualquier capa de la red.

$$z_j[\ell] = \sum_i w_{ij}[\ell] \cdot a_i[\ell-1] + b_j[\ell]$$

$$a_j[\ell] = g(z_j[\ell])$$

- $z_j[\ell]$  representa la combinación lineal que recibe la neurona  $j$  de la capa  $\ell$ .
- $a_j[\ell]$  es la activación o salida de esa neurona.
- $w_{ij}[\ell]$  es el peso que conecta la neurona  $i$  de la capa anterior con la neurona  $j$  de la capa actual.
- $b_j[\ell]$  es el sesgo o bias de la neurona.
- $g(\cdot)$  es la función de activación elegida para esa capa.

### 3.2.2. Capa de entrada y primera capa oculta.

Si retomamos el ejemplo de una red con dos entradas, dos capas ocultas de tres neuronas cada una y una capa de salida de tres neuronas, el recorrido de la información puede describirse de forma explícita. En la capa de entrada, las variables originales actúan como activaciones iniciales de la red:

$$a_1[1] = x_1$$

$$a_2[1] = x_2$$

A partir de ahí, la primera capa oculta combina esas dos entradas con sus pesos y sus sesgos correspondientes:

$$z_1[2] = w_{11}[2] \cdot x_1 + w_{21}[2] \cdot x_2 + b_1[2]$$

$$z_2[2] = w_{12}[2] \cdot x_1 + w_{22}[2] \cdot x_2 + b_2[2]$$

$$z_3[2] = w_{13}[2] \cdot x_1 + w_{23}[2] \cdot x_2 + b_3[2]$$

Después se aplica la función de activación a cada una de esas salidas lineales:

$$a_1[2] = g(z_1[2])$$

$$a_2[2] = g(z_2[2])$$

$$a_3[2] = g(z_3[2])$$

### 3.2.3. Segunda capa oculta.

Las activaciones obtenidas en la primera capa oculta pasan a la segunda capa oculta y se vuelven a combinar mediante nuevos pesos y sesgos. Esto ilustra una idea central del aprendizaje profundo: cada capa no trabaja ya con los datos crudos, sino con representaciones intermedias generadas por la capa anterior.

$$z_1[3] = w_{11}[3] \cdot a_1[2] + w_{21}[3] \cdot a_2[2] + w_{31}[3] \cdot a_3[2] + b_1[3]$$

$$z_2[3] = w_{12}[3] \cdot a_1[2] + w_{22}[3] \cdot a_2[2] + w_{32}[3] \cdot a_3[2] + b_2[3]$$

$$z_3[3] = w_{13}[3] \cdot a_1[2] + w_{23}[3] \cdot a_2[2] + w_{33}[3] \cdot a_3[2] + b_3[3]$$



$$a_1[3] = g(z_1[3])$$

$$a_2[3] = g(z_2[3])$$

$$a_3[3] = g(z_3[3])$$

### 3.2.4. Capa de salida y predicción final.

La capa de salida recibe como entrada las activaciones de la segunda capa oculta. Si en este ejemplo tenemos tres neuronas de salida, sus expresiones serían las siguientes:

$$z_1[4] = w_{11}[4] \cdot a_1[3] + w_{21}[4] \cdot a_2[3] + w_{31}[4] \cdot a_3[3] + b_1[4]$$

$$z_2[4] = w_{12}[4] \cdot a_1[3] + w_{22}[4] \cdot a_2[3] + w_{32}[4] \cdot a_3[3] + b_2[4]$$

$$z_3[4] = w_{13}[4] \cdot a_1[3] + w_{23}[4] \cdot a_2[3] + w_{33}[4] \cdot a_3[3] + b_3[4]$$

Las activaciones finales de la capa de salida son:

$$a_1[4] = g(z_1[4])$$

$$a_2[4] = g(z_2[4])$$

$$a_3[4] = g(z_3[4])$$

Por tanto, la predicción final de la red puede expresarse como  $\hat{y}_1 = a_1[4]$ ,  $\hat{y}_2 = a_2[4]$  y  $\hat{y}_3 = a_3[4]$ . Si se tratara de una clasificación multiclase, cada una de estas salidas representaría la puntuación o la probabilidad asociada a una clase.

### 3.2.5. Interpretación paso a paso del flujo de información.

El funcionamiento de un MLP puede entenderse como una cadena de transformaciones. En cada capa sucede siempre lo mismo: las neuronas reciben las salidas de la capa anterior, las multiplican por sus pesos, añaden un sesgo, calculan un valor  $z$  y aplican una función de activación para generar una nueva salida  $a$ . Esa nueva salida se convierte en la entrada de la siguiente capa.

Desde el punto de vista didáctico, esto permite explicar que una red multicapa no toma decisiones de una sola vez. Va construyendo una representación interna del problema capa a capa. Por eso puede aprender fronteras de decisión no lineales y relaciones mucho más complejas que las de un perceptrón simple.

### 3.2.6. Funciones de activación y entrenamiento del MLP.

En explicaciones introductorias es frecuente representar el MLP usando una función escalón o la función de Heaviside, porque enlaza bien con el perceptrón clásico. Sin embargo, en redes neuronales actuales las capas ocultas suelen utilizar activaciones como ReLU, tanh o sigmoide, ya que permiten calcular derivadas útiles durante el entrenamiento.

El entrenamiento del MLP se realiza mediante backpropagation y optimización por gradiente. Primero se hace una propagación hacia delante para obtener la predicción; después se calcula el error; a continuación ese error se distribuye hacia atrás capa a capa; y, por último, se actualizan los pesos y los sesgos para reducir la pérdida en la siguiente iteración.

## 3.3. Intuición: aprender representaciones intermedias.

Una forma intuitiva de entender un MLP es pensar que cada capa aprende una 'vista' diferente del problema. En un conjunto de variables de clientes, la primera capa puede combinar ingresos, gasto y antigüedad para detectar perfiles básicos. La siguiente capa puede combinar esos perfiles para identificar riesgo de abandono o probabilidad de compra.

En visión artificial sucede algo parecido: las capas iniciales detectan bordes y texturas; las intermedias, formas o partes; las finales, objetos completos. En lenguaje natural, ciertas capas detectan relaciones sintácticas o semánticas. El patrón común es siempre el mismo: las capas profundas transforman los datos crudos en representaciones cada vez más útiles para la tarea.

### 3.4. Papel de la no linealidad.

Si colocáramos varias capas lineales consecutivas sin funciones de activación no lineales, el resultado total seguiría siendo equivalente a una única transformación lineal. En otras palabras, aumentaríamos el número de pasos, pero no la capacidad expresiva real del modelo.

Por eso las funciones de activación son indispensables. La no linealidad rompe la simpleza de la suma ponderada y permite a la red aproximar relaciones complejas. Esta idea es central para comprender por qué el perceptrón multicapa supera las limitaciones del perceptrón simple.

**Ampliación.** Existe un resultado teórico conocido como teorema de aproximación universal: una red feedforward con al menos una capa oculta y una activación adecuada puede aproximar, con suficiente número de neuronas, una amplia familia de funciones continuas. Este teorema no dice que entrenar la red sea fácil ni que sea la opción óptima, pero sí justifica su enorme capacidad expresiva.

### 3.5. Capas de salida y tipos de tarea.

La forma de la capa de salida depende de la tarea. En clasificación binaria suele usarse una única neurona con activación sigmoide. En clasificación multiclase, una capa con tantas neuronas como clases y una activación softmax. En regresión, la salida suele ser lineal, porque queremos predecir un valor continuo y no una probabilidad.

Este punto es clave en la práctica: la arquitectura no se diseña de manera arbitraria, sino alineada con el tipo de salida que necesitamos. Un error típico en implementaciones de estudiantes es usar la activación de salida equivocada y, en consecuencia, una función de pérdida inadecuada.

### 3.6. Resumen comparativo entre perceptrón y MLP.

| Aspecto              | Perceptrón simple             | Perceptrón multicapa (MLP)                   |
|----------------------|-------------------------------|----------------------------------------------|
| Capas                | Entrada y salida              | Entrada, una o varias ocultas y salida       |
| Frontera de decisión | Lineal                        | Puede ser no lineal                          |
| Capacidad expresiva  | Baja                          | Alta                                         |
| Entrenamiento        | Regla simple de actualización | Backpropagation + optimización por gradiente |
| Uso actual           | Conceptual / educativo        | Muy frecuente en problemas reales            |

## 4. Funciones de activación: por qué son necesarias.

### 4.1. Qué hace una función de activación.

Después de calcular la combinación lineal  $z$ , la neurona necesita decidir cómo transformar ese valor en una salida útil. Esa transformación la realiza la función de activación. Según cuál elijamos, la neurona puede producir una salida binaria, una probabilidad, un valor centrado en cero o una activación no lineal muy sencilla pero eficaz.

En términos prácticos, la activación afecta a la estabilidad del entrenamiento, a la capacidad de la red para aprender patrones y a la forma de interpretar la salida. No es un detalle menor de implementación; es uno de los componentes que más influyen en el comportamiento del modelo.

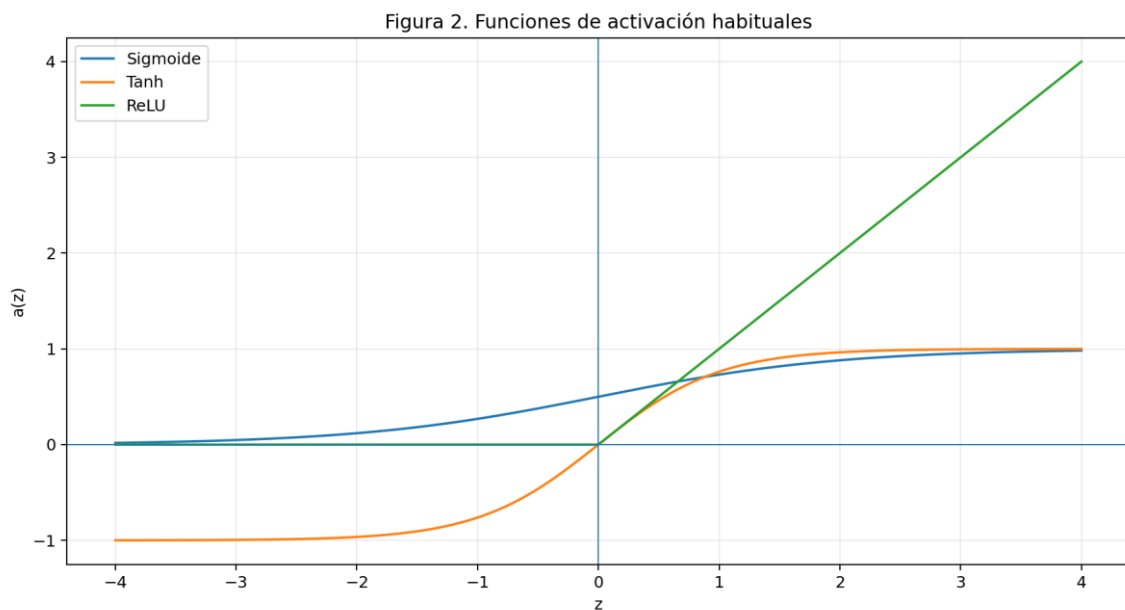


Figura 3. Curvas de varias funciones de activación habituales.

### 4.2. Función escalón y función signo.

La función escalón fue esencial en el perceptrón original. Su ventaja es la simplicidad: separa los casos en activado o no activado. Sin embargo, presenta un problema importante para el entrenamiento profundo: no es derivable de forma útil en la mayor parte de su dominio, lo que dificulta aplicar métodos de optimización basados en gradientes.

Por este motivo, aunque sigue siendo muy útil para explicar la lógica del perceptrón, no suele emplearse en redes profundas modernas.

### 4.3. Sigmoide.

La sigmoide transforma cualquier valor real en un número entre 0 y 1. Esta propiedad la hace muy adecuada para interpretar la salida como probabilidad en clasificación binaria. Si el resultado es 0.92, por ejemplo, podemos leerlo como una alta probabilidad de pertenecer a la clase positiva.

Su inconveniente principal es que, en valores muy altos o muy bajos, la pendiente se aplana y el gradiente se hace pequeño. Esto puede ralentizar el aprendizaje, especialmente en redes profundas. Aun así, sigue siendo una activación muy útil en la capa de salida de problemas binarios.

#### 4.4. Tanh.

La tangente hiperbólica produce salidas entre -1 y 1. Tiene una ventaja respecto a la sigmoide: está centrada en cero, lo que en ciertos escenarios facilita el aprendizaje. Históricamente se utilizó mucho en capas ocultas.

Su problema, sin embargo, se parece al de la sigmoide: también puede sufrir saturación y provocar gradientes pequeños cuando las activaciones se sitúan en regiones extremas.

#### 4.5. ReLU y sus variantes.

La ReLU, definida como  $\max(0, z)$ , se ha convertido en una de las activaciones más usadas en capas ocultas. Su éxito se debe a que es simple, barata computacionalmente y evita parte del problema del gradiente pequeño en la zona positiva. Gracias a ello, acelera el entrenamiento de muchas redes profundas.

El inconveniente de ReLU es que todas las entradas negativas se convierten en cero. Si una neurona queda permanentemente en esa región, puede dejar de aprender; a esto se le llama a veces 'neurona muerta'. Para suavizar ese problema se emplean variantes como Leaky ReLU o Parametric ReLU, que permiten una pequeña pendiente también en la zona negativa.

#### 4.6. Softmax.

La softmax se utiliza sobre todo en la capa de salida de clasificación multiclase. Toma un vector de puntuaciones y lo convierte en un conjunto de probabilidades que suman 1. De esta manera, si tenemos tres clases y la salida es [0.05, 0.15, 0.80], la red está indicando que considera mucho más probable la tercera clase.

Softmax no se suele usar en capas ocultas, sino como mecanismo final de interpretación probabilística cuando las clases son mutuamente excluyentes.

#### 4.7. Criterios prácticos para elegir la activación.

| Situación                          | Activación recomendada | Motivo                                                        |
|------------------------------------|------------------------|---------------------------------------------------------------|
| Clasificación binaria (salida)     | Sigmoide               | Entrega un valor interpretable como probabilidad.             |
| Clasificación multiclase (salida)  | Softmax                | Normaliza las puntuaciones en probabilidades que suman 1.     |
| Capas ocultas generales            | ReLU / Leaky ReLU      | Entrenamiento eficiente y buena respuesta en redes profundas. |
| Modelos antiguos o casos concretos | Tanh                   | Salida centrada en cero; útil en algunos escenarios.          |

**Idea clave.** Una red neuronal no aprende solo por tener muchas capas. Aprende bien cuando arquitectura, función de activación, salida y función de pérdida están alineadas con el problema.

## 5. Entrenamiento de redes neuronales.

### 5.1. Preparación de los datos: entrenamiento, validación y prueba.

Antes de entrenar una red debemos dividir el conjunto de datos. Lo habitual es reservar una parte para entrenamiento, otra para validación y otra para prueba. El conjunto de entrenamiento sirve para ajustar los pesos; el de validación ayuda a tomar decisiones durante el proceso, como elegir hiperparámetros o aplicar early stopping; el de prueba se usa al final para estimar el rendimiento real del modelo sobre datos no vistos.

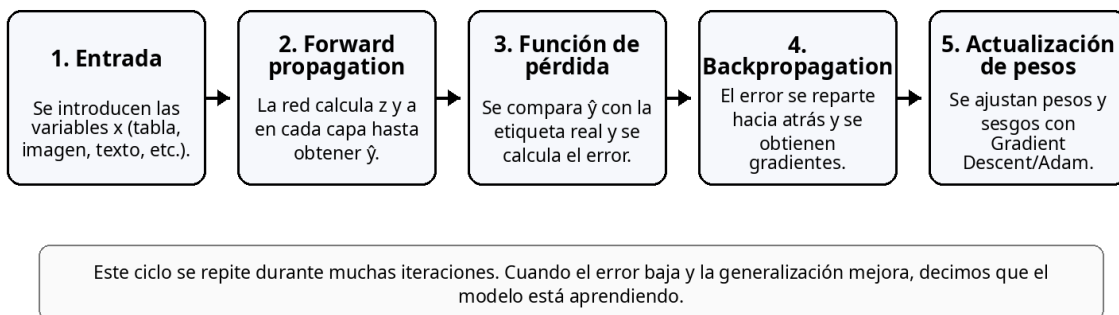
Esta división evita un error común: creer que el modelo es bueno solo porque memoriza los ejemplos con los que se ha entrenado. El objetivo no es recordar el conjunto de entrenamiento, sino generalizar a nuevos casos.

### 5.2. Forward propagation.

En la propagación hacia delante introducimos una muestra de entrada y calculamos, capa a capa, las activaciones de la red hasta obtener una predicción final. Cada capa aplica una transformación a la salida de la anterior. El proceso es determinista: con unos pesos fijos y una entrada concreta, la predicción siempre será la misma.

Desde un punto de vista funcional, forward propagation es la fase de inferencia interna del entrenamiento. Aún no estamos corrigiendo errores; estamos averiguando qué responde la red con los parámetros actuales.

**Figura 3. Flujo del entrenamiento de una red neuronal**



*Figura 4. Ciclo básico del entrenamiento: entrada, propagación, error, retropropagación y actualización de pesos.*

### 5.3. Función de pérdida o función de coste.

La función de pérdida mide la diferencia entre la predicción de la red y la respuesta correcta. Si el error es pequeño, la red está funcionando bien para ese ejemplo; si es grande, hay que corregir los parámetros. La pérdida es, por tanto, la señal que guía el aprendizaje.

En clasificación binaria se usa a menudo la binary cross-entropy. En clasificación multiclase, la categorical cross-entropy. En regresión, son frecuentes el error cuadrático medio (MSE) y el error absoluto medio (MAE). Escoger una pérdida coherente con la tarea es tan importante como escoger la activación de salida correcta.

### 5.4. Descenso por gradiente.

El descenso por gradiente es una estrategia de optimización. Imaginemos que la función de pérdida forma un paisaje con montañas y valles. Queremos encontrar una zona baja, es decir, una combinación de pesos

que reduzca el error. Para ello calculamos el gradiente, que nos indica en qué dirección crece más la pérdida. Si queremos minimizarla, debemos movernos en la dirección contraria.

Cada actualización de pesos sigue la idea general:  $\text{nuevo\_peso} = \text{peso\_actual} - \text{tasa\_de\_aprendizaje} \times \text{gradiente}$ . La tasa de aprendizaje o learning rate controla el tamaño del paso. Si es demasiado grande, podemos saltarnos el mínimo o hacer el entrenamiento inestable; si es demasiado pequeña, el aprendizaje será muy lento.

### 5.5. Batch, mini-batch y época.

Una época es un recorrido completo por todo el conjunto de entrenamiento. Un batch es un subconjunto de muestras que la red procesa antes de actualizar los pesos. Cuando el tamaño del batch es pequeño o intermedio, hablamos de mini-batch gradient descent, que es el enfoque más habitual en deep learning.

Trabajar por mini-batches tiene ventajas prácticas: reduce el consumo de memoria, introduce una cierta variabilidad beneficiosa en el entrenamiento y aprovecha mejor el paralelismo del hardware.

### 5.6. Backpropagation.

La retropropagación es el mecanismo que permite repartir el error desde la salida hacia las capas anteriores. Una vez calculada la pérdida, backpropagation determina cuánto ha contribuido cada peso al error final y obtiene los gradientes necesarios para actualizarlo.

Lo esencial para el alumnado de ASIR es entender que la red aprende de sus errores porque puede medir la contribución de cada parámetro. Para el nivel de Máster de IA conviene añadir que este reparto se realiza aplicando sistemáticamente la regla de la cadena a funciones compuestas. La salida depende de las activaciones intermedias; las activaciones intermedias dependen de otras anteriores; y así sucesivamente hasta llegar a cada peso.

En esta unidad basta con comprender la lógica general de la retropropagación. La derivación formal de las expresiones de backpropagation, con las fórmulas de los gradientes y del término de error local  $\delta$ , se desarrolla en el Anexo 14.2 para el alumnado que necesite una profundización matemática mayor.

**Ampliación.** Si la pérdida se denota por  $L$  y un peso concreto por  $w$ , backpropagation calcula  $\partial L / \partial w$ . Esta derivada no se obtiene de forma aislada, sino encadenando derivadas locales de cada capa. El algoritmo es eficiente porque reutiliza cálculos intermedios y propaga el error de la última capa hacia las anteriores.

### 5.7. Optimizadores modernos: Adam y otros.

El descenso por gradiente estándar es una idea base, pero en la práctica se emplean optimizadores más sofisticados. Adam es uno de los más populares porque adapta el tamaño de los pasos a partir de estimaciones del primer y del segundo momento de los gradientes. En lenguaje menos formal: intenta combinar velocidad y estabilidad.

Otros optimizadores, como RMSProp o SGD con momentum, siguen la misma lógica de mejorar el recorrido por la superficie de error. Para un curso de ASIR basta con entender que son variantes del proceso de ajuste; para un Máster de IA interesa analizar en qué condiciones convergen mejor o peor.

## 5.8. Hiperparámetros fundamentales.

| Hiperparámetro     | Qué controla                     | Consecuencia de un mal ajuste                  |
|--------------------|----------------------------------|------------------------------------------------|
| Learning rate      | Tamaño del paso de actualización | Entrenamiento inestable o demasiado lento      |
| Número de capas    | Profundidad del modelo           | Capacidad insuficiente o exceso de complejidad |
| Número de neuronas | Anchura de cada capa             | Subajuste o sobreajuste                        |
| Batch size         | Muestras por actualización       | Ruido excesivo o alto consumo de memoria       |
| Épocas             | Número de recorridos completos   | Subentrenamiento o sobreentrenamiento          |

## 5.9. Ejemplo sencillo de entrenamiento.

Supongamos una red pequeña que intenta clasificar incidencias de servidor como 'crítica' o 'no crítica' usando variables como uso de CPU, consumo de memoria y número de errores por minuto. En la primera pasada, la red predice mal muchos ejemplos porque sus pesos se inicializan casi al azar. Tras calcular la pérdida y aplicar backpropagation, los pesos se ajustan. En las siguientes épocas, la red va mejorando porque encuentra patrones que relacionan determinadas combinaciones de variables con incidencias críticas.

Este ejemplo conecta bien con ASIR porque muestra una aplicación cercana al ámbito de sistemas. Lo importante es observar que el aprendizaje no se basa en reglas manuales, sino en ajustar automáticamente un gran número de parámetros a partir de datos históricos etiquetados.

## 6. Problemas frecuentes en el entrenamiento.

### 6.1. Underfitting o subajuste.

Hay subajuste cuando el modelo no consigue capturar ni siquiera la estructura básica del problema. En ese caso, tanto el error de entrenamiento como el de validación son altos. Puede deberse a una red demasiado simple, a un número insuficiente de épocas, a una mala representación de los datos o a un preprocesado inadecuado.

Desde el punto de vista didáctico, el subajuste recuerda al alumnado que un modelo muy simple también puede ser un problema. No solo existe el riesgo de que una red sea demasiado compleja; también puede quedarse corta.

### 6.2. Overfitting o sobreajuste.

Hay sobreajuste cuando el modelo aprende demasiado bien los detalles del conjunto de entrenamiento, incluidos el ruido y las particularidades accidentales, pero pierde capacidad para generalizar. El síntoma típico es que la pérdida de entrenamiento sigue bajando mientras la de validación deja de mejorar o incluso empeora.

En términos simples, la red memoriza en lugar de aprender de forma robusta. Este es uno de los conceptos más importantes del RA5, porque reconocer una red entrenada implica detectar precisamente estos comportamientos.

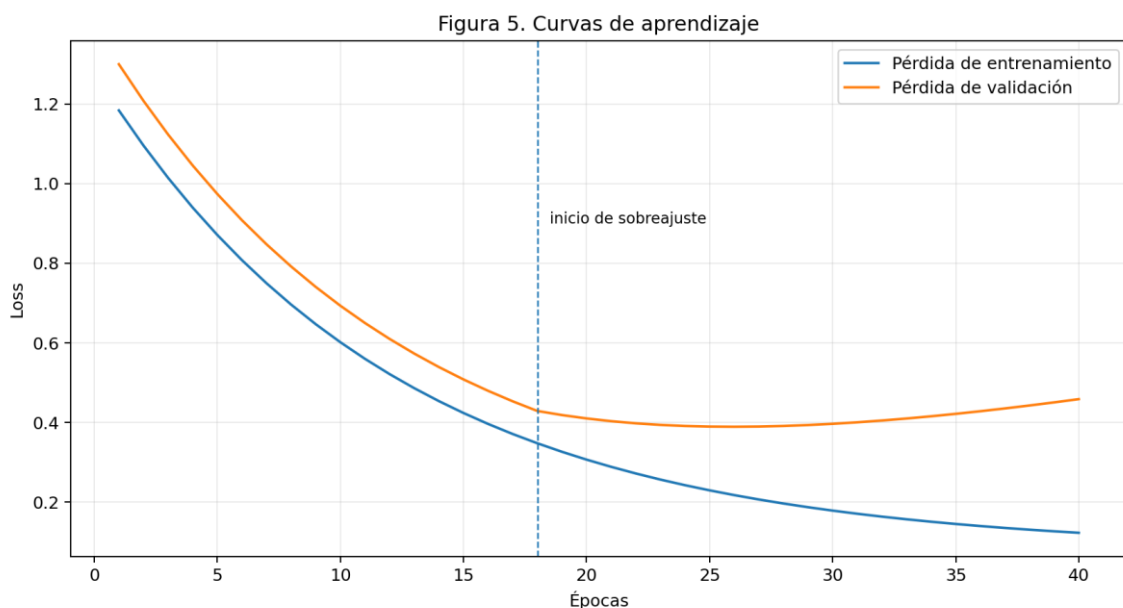


Figura 5. Curvas de aprendizaje: cuando la validación deja de mejorar mientras el entrenamiento sigue bajando, suele aparecer el sobreajuste.

### 6.3. Técnicas para reducir el sobreajuste.

- Recoger más datos o ampliar artificialmente el conjunto con técnicas de data augmentation cuando sea posible.
- Reducir la complejidad del modelo: menos capas, menos neuronas o arquitectura más contenida.
- Aplicar regularización L1 o L2 para penalizar pesos excesivamente grandes.
- Usar dropout, que desactiva de forma aleatoria una parte de las neuronas durante el entrenamiento.



- Aplicar early stopping, es decir, detener el entrenamiento cuando la validación deja de mejorar.
- Mejorar el preprocesado y la calidad de las variables o características.

## 6.4. Desaparición y explosión del gradiente.

En redes profundas, los gradientes pueden volverse extremadamente pequeños al propagarse hacia atrás, dificultando el aprendizaje en capas iniciales. A esto se le llama desaparición del gradiente. El problema contrario es la explosión del gradiente, cuando las derivadas crecen demasiado y vuelven inestable el entrenamiento.

Las arquitecturas modernas, ciertas funciones de activación, las inicializaciones adecuadas y los optimizadores avanzados ayudan a controlar estos problemas. En RNN clásicas, la desaparición del gradiente fue uno de los motivos que impulsaron el desarrollo de LSTM y GRU.

## 6.5. Normalización y escalado.

Muchas redes entrenan mejor si las entradas están escaladas de forma coherente. Si una variable toma valores entre 0 y 1 y otra entre 0 y 100000, la optimización puede volverse más difícil. Normalizar o estandarizar las variables ayuda a estabilizar el aprendizaje y acelera la convergencia.

Este paso es especialmente importante en datos tabulares y en cualquier pipeline profesional. No es un detalle accesorio: forma parte del diseño del sistema de IA.

**Idea clave.** Diagnosticar problemas de entrenamiento es parte del trabajo real con redes neuronales. No basta con ejecutar `fit()` y aceptar la salida: hay que observar curvas, métricas, distribución de errores y coherencia del modelo con el problema.

# 7. Tipos de redes neuronales y campos de aplicación.

## 7.1. Redes totalmente conectadas (MLP).

Las redes totalmente conectadas son la forma más general de red neuronal feedforward. Cada neurona de una capa se conecta con todas las de la siguiente. Suelen aplicarse a problemas de clasificación o regresión sobre datos tabulares: scoring de riesgo, predicción de abandono de clientes, clasificación de incidencias, detección de fraude o análisis de sensores.

Su ventaja principal es la versatilidad. Su principal limitación es que no aprovechan estructuras especiales de los datos. En una imagen, por ejemplo, tratar cada píxel como una entrada independiente ignora la vecindad espacial. En una secuencia de texto, tampoco explotan el orden temporal de manera natural.

## 7.2. Redes convolucionales (CNN).

Las CNN están diseñadas para datos con estructura espacial, especialmente imágenes. En lugar de conectar todo con todo desde el principio, utilizan filtros o kernels que recorren localmente la entrada. Cada filtro detecta patrones como bordes, texturas o formas simples. A medida que la información avanza por la red, los patrones detectados se vuelven más complejos.

Esta estrategia reduce drásticamente el número de parámetros en comparación con una red totalmente conectada equivalente y aprovecha mejor la naturaleza local de las imágenes. Por eso las CNN han sido

fundamentales en clasificación de imágenes, detección de objetos, reconocimiento médico, visión industrial y vigilancia.

### 7.3. Redes recurrentes (RNN, LSTM y GRU).

Las redes recurrentes fueron diseñadas para datos secuenciales. Su idea central es que la salida de un paso no depende solo de la entrada actual, sino también de un estado interno que resume lo visto anteriormente. Eso las hace apropiadas para texto, señales temporales, series financieras o registros de eventos.

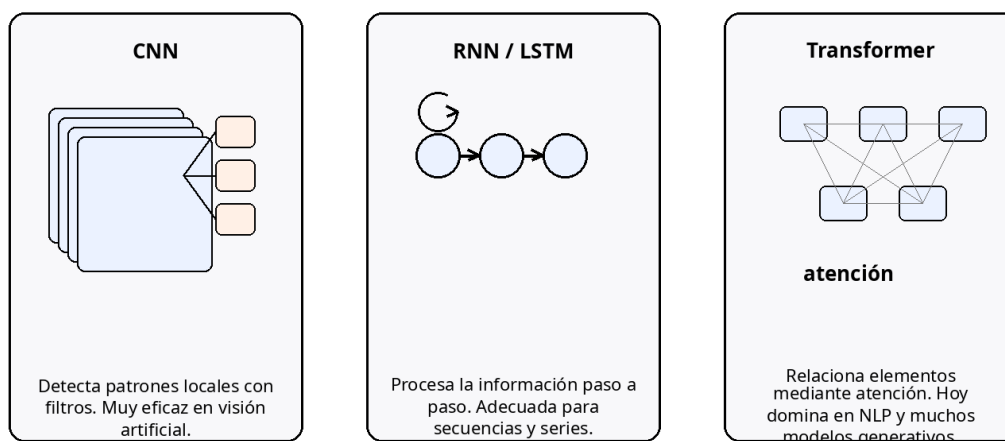
LSTM y GRU son variantes que incorporan mecanismos de compuertas para conservar o descartar información de manera más eficaz. Gracias a ello pueden modelar dependencias a más largo plazo y mitigar algunos problemas de las RNN clásicas.

### 7.4. Transformers (ampliación recomendada).

Los transformers se han convertido en una arquitectura dominante en procesamiento del lenguaje natural y, cada vez más, en visión y aprendizaje multimodal. Su mecanismo central es la atención, que permite modelar relaciones entre elementos de una secuencia sin procesarlos estrictamente uno detrás de otro.

Desde la perspectiva didáctica, conviene presentar los transformers como una ampliación lógica de la evolución de las redes. No sustituyen el valor pedagógico de MLP, CNN o RNN, pero ayudan a entender por qué los modelos generativos actuales tienen tanta capacidad.

**Figura 8. Tres familias de arquitecturas profundas**



### 7.5. Qué arquitectura encaja mejor con cada tipo de dato.

| Tipo de dato / tarea | Arquitectura habitual                       | Comentario didáctico                                          |
|----------------------|---------------------------------------------|---------------------------------------------------------------|
| Datos tabulares      | MLP, pero también modelos clásicos          | A menudo Random Forest o Gradient Boosting compiten muy bien. |
| Imágenes             | CNN o modelos visuales preentrenados        | Aprovechan la estructura espacial.                            |
| Texto y lenguaje     | RNN/LSTM/GRU o transformers                 | Importa el contexto y el orden de la secuencia.               |
| Series temporales    | RNN/LSTM/GRU, TCN o transformers temporales | La dependencia entre instantes es esencial.                   |

|       |                                               |                                               |
|-------|-----------------------------------------------|-----------------------------------------------|
| Audio | CNN sobre espectrogramas o redes secuenciales | Se trabaja con patrones locales y temporales. |
|-------|-----------------------------------------------|-----------------------------------------------|

**Ampliación.** Merece la pena comparar también complejidad computacional, número de parámetros, consumo de memoria y posibilidad de transferencia de aprendizaje. Por ejemplo, en visión actual muchas soluciones no se entrenan desde cero, sino a partir de modelos preentrenados.

## 8. Comparación con otros métodos de inteligencia artificial.

### 8.1. Por qué comparar y no solo describir.

El RA5 exige comparar redes neuronales con otros métodos de IA. Esta comparación es imprescindible porque en entornos profesionales rara vez se plantea una única solución posible. Normalmente se prueban varias familias de modelos y se selecciona la que ofrezca mejor equilibrio entre precisión, coste, interpretabilidad y facilidad de despliegue.

Por eso una buena formación no consiste en 'enamorarse' de las redes neuronales, sino en entender qué ofrecen frente a alternativas sólidas y cuándo dejan de ser la mejor elección.

### 8.2. Regresión logística.

La regresión logística es un modelo clásico muy eficaz en clasificación binaria. A pesar de su nombre, es un clasificador. Su gran ventaja es la simplicidad: se entrena rápido, requiere pocos recursos y sus coeficientes permiten cierta interpretación sobre la influencia de cada variable.

Cuando el problema es aproximadamente lineal y los datos son tabulares, una regresión logística bien preparada puede ser una referencia excelente. De hecho, en muchos proyectos conviene usarla como baseline antes de probar modelos más complejos.

### 8.3. Árboles de decisión.

Los árboles de decisión dividen el espacio de datos mediante preguntas del tipo 'si la variable X supera cierto umbral, ve a la izquierda; si no, a la derecha'. Son muy interpretables porque permiten reconstruir el camino de decisión paso a paso.

Su principal inconveniente es que un árbol individual puede ser inestable y propenso al sobreajuste. Aun así, son muy valiosos cuando la explicabilidad es prioritaria, por ejemplo en entornos educativos, financieros o de soporte a decisión.

### 8.4. Random Forest y ensamblados.

Random Forest combina muchos árboles entrenados sobre subconjuntos de datos y variables. El resultado suele ser mucho más robusto que un árbol individual. En datos tabulares, Random Forest y otros ensamblados pueden ofrecer rendimientos excelentes con menos ajuste fino que una red neuronal.

Desde un punto de vista práctico, esta familia de modelos es una de las rivales más fuertes de las redes neuronales en problemas no visuales y no secuenciales.

## 8.5. Máquinas de vectores de soporte (SVM).

Las SVM buscan una frontera que separe las clases maximizando el margen entre ellas. Con kernels adecuados pueden modelar relaciones no lineales. Funcionan bien en conjuntos de datos pequeños o medianos y de alta dimensión, pero escalan peor cuando el volumen de datos crece mucho.

En proyectos docentes, las SVM son interesantes porque muestran otra forma muy distinta de construir clasificadores potentes sin recurrir a arquitecturas profundas.

## 8.6. k-NN.

El algoritmo k vecinos más cercanos clasifica una muestra observando qué ejemplos similares tiene alrededor. Es muy fácil de entender e implementar, pero el coste de predicción puede ser alto porque necesita comparar la nueva muestra con muchas anteriores. También depende mucho de la escala de los datos y de la métrica elegida.

Didácticamente es útil porque ejemplifica un enfoque basado en semejanza local, muy diferente de la lógica paramétrica de una red neuronal.

## 8.7. Tabla comparativa global.

| Método              | Ventajas principales                  | Limitaciones                           | Cuándo suele ser buena opción              |
|---------------------|---------------------------------------|----------------------------------------|--------------------------------------------|
| Regresión logística | Simple, rápida, interpretable         | Supone relaciones casi lineales        | Clasificación binaria tabular y baseline   |
| Árbol de decisión   | Muy explicable                        | Puede sobreajustar                     | Procesos con necesidad de reglas claras    |
| Random Forest       | Robusto y eficaz en tabular           | Menos interpretable que un árbol       | Datos tabulares con pocas transformaciones |
| SVM                 | Buen rendimiento en datasets pequeños | Escala peor con muchos datos           | Problemas medianos y alta dimensión        |
| k-NN                | Conceptualmente sencillo              | Predicción costosa y sensible a escala | Escenarios docentes o problemas simples    |
| Red neuronal        | Muy flexible y potente                | Más datos, más ajuste y más coste      | Patrones complejos, imágenes, texto, audio |

## 8.8. Comparación en lenguaje de aula.

- Si el problema es pequeño y quiero una solución rápida y explicable, empiezo por modelos clásicos.
- Si tengo imágenes, audio o texto complejo, las redes neuronales pasan a ser candidatas naturales.
- Si dispongo de pocos datos, una red profunda puede no ser la mejor primera elección.
- Si necesito máxima interpretabilidad, un árbol o una regresión logística suelen ser preferibles.
- Si busco la mejor precisión posible y tengo suficientes datos y hardware, las redes profundas pueden marcar la diferencia.

**Idea clave.** Comparar no significa solo decir cuál tiene más accuracy. Comparar implica analizar costes, datos necesarios, tiempo de entrenamiento, facilidad de despliegue, explicabilidad y riesgos de sobreajuste.

## 9. Cómo elegir el modelo adecuado según el problema.

### 9.1. Criterios de decisión.

Elegir un modelo es una decisión técnica, no una preferencia estética. Para decidir bien conviene responder a varias preguntas: ¿qué tipo de datos tengo?, ¿cuántos ejemplos poseo?, ¿qué nivel de explicabilidad exige el contexto?, ¿cuánto tiempo y hardware tengo para entrenar?, ¿importa más la precisión máxima o la rapidez de despliegue?, ¿puedo reutilizar un modelo preentrenado?

Cuando el alumnado aprende a formular estas preguntas, deja de ver la IA como un catálogo de algoritmos aislados y empieza a entenderla como un proceso de ingeniería.

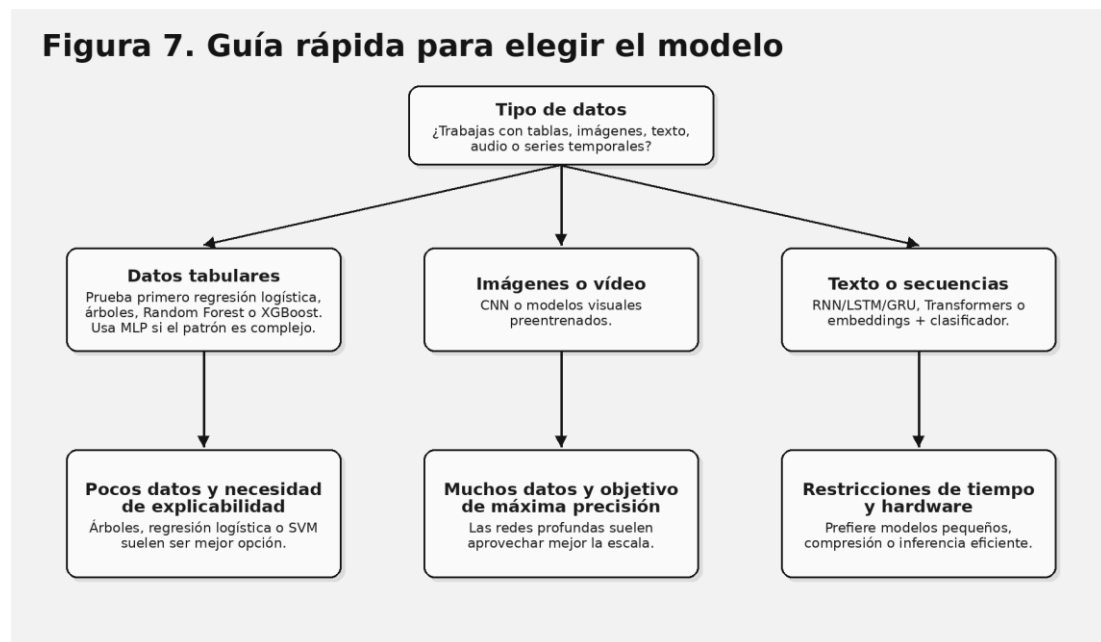


Figura 7. Secuencia de preguntas útil para orientar la selección del modelo.

### 9.2. Selección según el tipo de dato.

| Escenario                       | Modelo que conviene probar primero                              | Justificación                                                                                              |
|---------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Cientes, ventas, logs tabulares | Regresión logística / árboles / Random Forest / MLP             | Los datos tabulares suelen resolverse bien con modelos clásicos; el MLP entra si hay interacción compleja. |
| Clasificación de imágenes       | CNN                                                             | Aprovecha la estructura espacial de la imagen.                                                             |
| Análisis de correo o texto      | Embeddings + clasificador, RNN o transformer                    | El orden y el contexto del lenguaje son fundamentales.                                                     |
| Series temporales               | Modelos clásicos de series, RNN/LSTM/GRU o transformer temporal | Interesa modelar dependencias en el tiempo.                                                                |

### 9.3. Selección según el tamaño del conjunto de datos.

Con pocos datos, los modelos muy complejos corren más riesgo de sobreajuste. En esos casos suele ser sensato empezar por métodos clásicos y usar una red neuronal solo si el problema lo justifica o si podemos transferir conocimiento desde un modelo preentrenado.

Con grandes volúmenes de datos, las redes profundas suelen beneficiarse más de la escala. Cuantos más ejemplos representativos reciben, mejor pueden ajustar sus numerosos parámetros sin memorizar el ruido de forma excesiva.

### 9.4. Selección según la explicabilidad.

No siempre gana el modelo más preciso. En algunos entornos es imprescindible poder explicar por qué se ha tomado una decisión. Si una institución educativa, financiera o sanitaria necesita justificar una clasificación, un árbol, una regresión o un modelo más interpretable puede ser preferible a una red profunda opaca.

Esto no significa que las redes sean inútiles en contextos sensibles, sino que exigen técnicas adicionales de interpretabilidad y una validación más rigurosa.

### 9.5. Selección según el coste de entrenamiento e inferencia.

Entrenar una red grande puede requerir GPU, tiempo y consumo energético elevado. Si el proyecto es pequeño o debe ejecutarse en hardware limitado, quizá interese un modelo más ligero. Lo mismo ocurre en inferencia: un sistema que responde en tiempo real en un dispositivo modesto no puede depender siempre de un modelo enorme.

Por eso el criterio de eficiencia también forma parte de la elección del modelo. En un Máster de IA conviene relacionar esta idea con compresión de modelos, cuantización o distillation.

### 9.6. Procedimiento recomendado de trabajo.

- Definir claramente la tarea y la métrica principal.
- Construir un baseline simple y razonable.
- Probar una o dos alternativas adicionales con distinta filosofía.
- Comparar resultados con validación adecuada, no solo con el entrenamiento.
- Analizar errores y decidir si compensa aumentar complejidad.
- Documentar la elección final con criterios técnicos y no solo con intuiciones.

**Ejemplo de razonamiento correcto.** Si voy a clasificar incidencias a partir de variables tabulares y tengo 5000 ejemplos, empezaré por regresión logística, árbol y Random Forest. Solo pasaré a un MLP si los modelos anteriores no capturan bien el patrón o si observo interacciones complejas entre variables.

## 10. Cómo reconocer una red entrenada y valorar su rendimiento.

### 10.1. No basta con mirar la accuracy.

Muchos estudiantes se fijan únicamente en la accuracy, es decir, en el porcentaje total de aciertos. Sin embargo, esa métrica puede ser engañosa. Imaginemos un problema donde el 95 % de los casos pertenece

a la clase 'normal' y solo el 5 % a la clase 'crítica'. Un modelo que prediga siempre 'normal' tendría 95 % de accuracy y, sin embargo, sería inútil.

Por eso debemos interpretar el rendimiento con varias métricas y teniendo en cuenta el contexto del problema.

## 10.2. Matriz de confusión.

La matriz de confusión resume cuántos casos se han clasificado correctamente y cuántos se han confundido. En clasificación binaria aparecen cuatro cantidades: verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos. Esta matriz permite ver el comportamiento real del modelo, no solo una cifra agregada.

En un sistema de alertas críticas, por ejemplo, los falsos negativos son especialmente peligrosos porque significan que el sistema no detecta una incidencia grave. En cambio, en otros contextos pueden preocupar más los falsos positivos porque generan alarmas innecesarias.

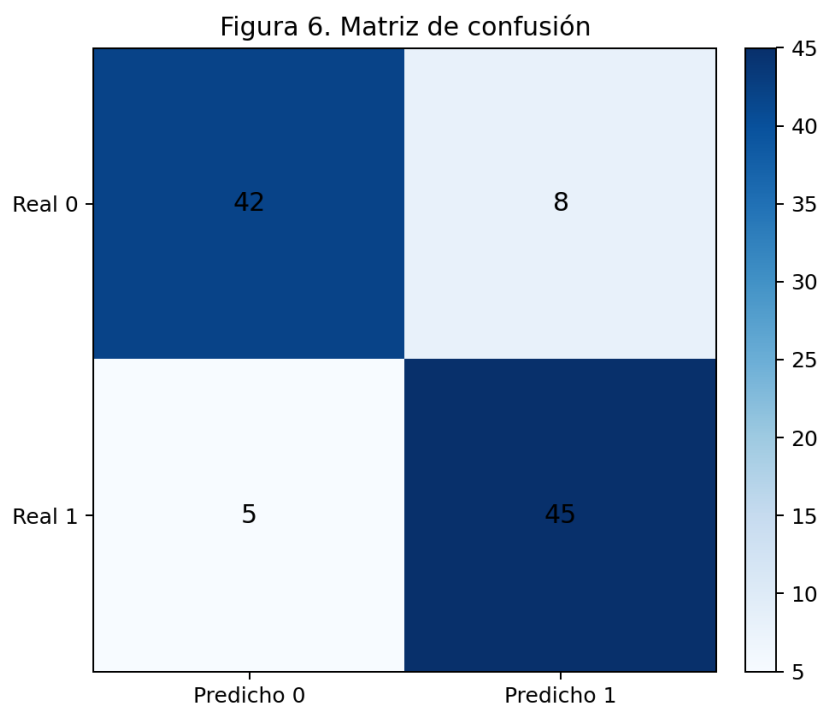


Figura 8. Ejemplo de matriz de confusión para una clasificación binaria.

## 10.3. Precisión, recall y F1-score.

La precisión indica de todos los casos que el modelo ha marcado como positivos, cuántos eran realmente positivos. El recall o sensibilidad indica de todos los positivos reales, cuántos ha detectado el modelo. El F1-score combina precisión y recall y es muy útil cuando interesa equilibrar ambos aspectos.

Estas métricas ayudan al alumnado a razonar más allá del porcentaje de aciertos. Un modelo puede tener buena precisión y mal recall, o al revés. Lo importante es interpretar qué significa eso para la aplicación concreta.

| Métrica   | Pregunta que responde                                     | Cuándo importa especialmente                |
|-----------|-----------------------------------------------------------|---------------------------------------------|
| Accuracy  | ¿Qué porcentaje total de aciertos tengo?                  | Clases equilibradas y análisis general.     |
| Precisión | De lo que he marcado como positivo, ¿cuánto era correcto? | Cuando el coste del falso positivo es alto. |

|          |                                                 |                                             |
|----------|-------------------------------------------------|---------------------------------------------|
| Recall   | De los positivos reales, ¿cuántos he detectado? | Cuando el coste del falso negativo es alto. |
| F1-score | ¿Cómo equilibrio precisión y recall?            | Cuando ambas dimensiones son relevantes.    |

### 10.4. Curvas de aprendizaje.

Las curvas de aprendizaje representan cómo evoluciona la pérdida o una métrica a lo largo de las épocas. Son una herramienta potentísima para reconocer si la red está aprendiendo, si se ha estancado o si está sobreajustando.

Si la pérdida de entrenamiento baja y la de validación también, vamos bien. Si la de entrenamiento baja pero la de validación empeora, estamos ante un síntoma de sobreajuste. Si ambas se mantienen altas, probablemente hay subajuste.

### 10.5. Reconocimiento práctico de modelos entrenados.

- Modelo bien ajustado: entrenamiento y validación mejoran de forma coherente; las métricas tienen sentido en test.
- Modelo sobreajustado: entrenamiento muy bueno, validación peor y pérdida de generalización.
- Modelo subajustado: ni entrenamiento ni validación alcanzan un rendimiento razonable.
- Modelo sesgado por datos: errores sistemáticos en una subclase, aunque la métrica global parezca buena.
- Modelo inestable: resultados muy distintos entre ejecuciones o alta sensibilidad a pequeños cambios.

### 10.6. Ampliación: ROC-AUC y calibración.

En un Máster de IA conviene introducir también la curva ROC y el área bajo la curva (AUC), especialmente en clasificación binaria con distintos umbrales posibles. Del mismo modo, es útil hablar de calibración: una red no solo debe ordenar bien los casos, sino estimar probabilidades realistas.

Por ejemplo, si un modelo asigna muchas predicciones de 0.90, debería ocurrir que aproximadamente nueve de cada diez casos con esa confianza sean realmente positivos.

**Idea clave.** Reconocer una red entrenada significa interpretar salidas, métricas, curvas y errores. No es memorizar nombres de algoritmos, sino leer el comportamiento del modelo con criterio técnico.



## 11. Ejemplos guiados paso a paso.

### 11.1. Ejemplo 1 - Perceptrón para una decisión binaria simple.

Supongamos que queremos clasificar si una alerta de monitorización debe escalarse manualmente. Usamos dos variables:  $x_1$  = CPU alta (0 o 1) y  $x_2$  = errores críticos presentes (0 o 1). Queremos que la salida sea 1 cuando ambas condiciones se cumplan.

Como vimos antes, un perceptrón con pesos  $w_1 = 1$ ,  $w_2 = 1$  y sesgo  $b = -1.5$  resuelve el problema AND. Este ejemplo es muy apropiado para una primera práctica porque permite calcular a mano cada salida y relacionar la decisión con la geometría de la frontera lineal.

### 11.2. Ejemplo 2 - MLP para clasificación tabular.

Ahora imaginemos un conjunto de clientes de una plataforma con variables numéricas y categóricas transformadas: frecuencia de acceso, tiempo de sesión, número de incidencias, antigüedad y gasto medio. El objetivo es predecir si el cliente abandonará el servicio en el próximo mes.

En este caso podemos construir un MLP sencillo con una capa de entrada, dos capas ocultas densas con activación ReLU y una capa de salida sigmoide. El flujo sería: normalizar variables, dividir los datos en train/validation/test, entrenar con binary cross-entropy, observar curvas de aprendizaje y ajustar hiperparámetros.

Lo interesante aquí no es solo ejecutar el modelo, sino compararlo con alternativas como regresión logística y Random Forest. Si el MLP mejora claramente en validación y test, habrá justificación para su uso. Si la mejora es mínima, quizá no compense la complejidad añadida.

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Dropout

model = Sequential([
    Dense(64, activation='relu', input_shape=(n_features,)),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

### 11.3. Qué debe interpretar el alumnado en el ejemplo del MLP.

- Que la salida sigmoide es coherente con una clasificación binaria.
- Que dropout se introduce para mejorar la generalización.
- Que el número de neuronas y capas no es arbitrario: se escoge como hipótesis de partida.
- Que accuracy por sí sola no basta; hay que mirar también precisión, recall, F1 y matriz de confusión.
- Que el modelo debe compararse con métodos clásicos para justificar su uso.

### 11.4. Ejemplo 3 - CNN para clasificación de imágenes.

Supongamos una tarea de clasificación de imágenes de piezas industriales correctas y defectuosas. Una CNN aplicará filtros locales sobre la imagen para detectar patrones visuales relevantes. Las primeras capas aprenderán bordes, contrastes o texturas; las siguientes, configuraciones más complejas que distinguen un defecto real de una variación inocua.

En un entorno profesional rara vez se empieza desde cero. Lo habitual es usar transferencia de aprendizaje a partir de una CNN preentrenada y reajustarla con nuestro conjunto de imágenes.

### 11.5. Ejemplo 4 - RNN/LSTM para secuencias.

Consideremos una serie temporal de carga del sistema medida cada minuto. Si queremos anticipar un pico futuro, el orden de las observaciones importa. Una RNN o una LSTM puede procesar la secuencia teniendo en cuenta la memoria de estados anteriores.

En este escenario, el modelo no debe ver cada instante como una fila aislada, sino como parte de una evolución temporal. Esto ilustra por qué la estructura del dato condiciona la arquitectura adecuada.

### 11.6. Ejemplo 5 - Comparación directa de modelos.

| Modelo              | Accuracy | F1   | Interpretabilidad | Comentario                                      |
|---------------------|----------|------|-------------------|-------------------------------------------------|
| Regresión logística | 0.83     | 0.78 | Alta              | Buen baseline; entrenamiento rápido.            |
| Árbol de decisión   | 0.80     | 0.74 | Muy alta          | Fácil de explicar, pero menos robusto.          |
| Random Forest       | 0.88     | 0.84 | Media             | Gran opción en tabular.                         |
| MLP                 | 0.90     | 0.86 | Baja              | Ligeramente mejor, pero más costoso de ajustar. |

En este ejemplo, el MLP es el mejor en rendimiento puro, pero la diferencia respecto a Random Forest no es enorme. Si el proyecto exige rapidez de despliegue y explicación razonable, Random Forest podría ser la opción más equilibrada. Si el objetivo es exprimir al máximo la precisión y ya se dispone de pipeline de entrenamiento, el MLP puede justificarse.

**Pregunta de examen.** Si una red neuronal obtiene 92 % de accuracy y un Random Forest obtiene 90 %, no se debe elegir automáticamente la red. Hay que analizar si esa mejora compensa la pérdida de interpretabilidad, el coste computacional y el riesgo de sobreajuste.

## 12. Ampliación.

### 12.1. Regla de la cadena y gradientes.

Si la pérdida final depende de muchas capas encadenadas, el cálculo de un gradiente concreto requiere aplicar la regla de la cadena. Para una capa  $l$ , el interés suele centrarse en obtener  $\partial L / \partial W[l]$  y  $\partial L / \partial b[l]$ . Intuitivamente, queremos medir cómo cambiaría la pérdida si perturbáramos ligeramente cada peso o sesgo.

Backpropagation organiza este cálculo de manera eficiente. Primero obtiene el error local de la capa de salida. Después va propagando ese error hacia atrás multiplicándolo por derivadas locales de activaciones y pesos. El resultado es una secuencia de gradientes que permiten actualizar todos los parámetros.

### 12.2.1. Idea general y notación de una neurona.

Cuando una red neuronal comienza a entrenarse, sus pesos y sesgos se inicializan normalmente con valores aleatorios. Por eso, al principio, la salida de la red rara vez coincide con el valor esperado. La función de coste mide esa diferencia y el algoritmo de backpropagation nos permite calcular cómo cambia el error cuando modificamos cada parámetro.

Para una neurona  $j$  de la capa  $\ell$  utilizaremos la notación siguiente:

$$z_j[\ell] = \sum_i w_{ij}[\ell] \cdot a_i[\ell-1] + b_j[\ell]$$

$$a_j[\ell] = g(z_j[\ell])$$

Si denotamos por  $L$  la última capa y usamos una función de coste cuadrática para una muestra, el error total en la salida puede escribirse como:

$$C = 1/2 \cdot \sum_j (a_j[L] - y_j)^2$$

Nuestro objetivo es obtener las derivadas parciales del coste con respecto a todos los pesos y sesgos, porque esas derivadas indican cuánto cambia el error si alteramos ligeramente un parámetro concreto.

### 12.2.2. Derivadas en la última capa.

En la última capa, un peso no influye en el coste de forma directa. Antes de afectar al error total, pasa por varias etapas: modifica la suma ponderada  $z$ , esa suma condiciona la activación  $a$  y, finalmente, la activación interviene en la función de coste. Por eso debemos aplicar la regla de la cadena.

$$\partial C / \partial w_{ij}[L] = (\partial C / \partial a_j[L]) \cdot (\partial a_j[L] / \partial z_j[L]) \cdot (\partial z_j[L] / \partial w_{ij}[L])$$

$$\partial C / \partial b_j[L] = (\partial C / \partial a_j[L]) \cdot (\partial a_j[L] / \partial z_j[L]) \cdot (\partial z_j[L] / \partial b_j[L])$$

La clave es que no se calcula toda la derivada de una sola vez, sino que se descompone en derivadas locales más sencillas y después se combinan.

### 12.2.3. Derivadas intermedias y término de error local $\delta$ .

Si derivamos la función de coste cuadrática con respecto a la activación de la neurona  $j$  en la última capa, obtenemos:

$$\partial C / \partial a_j[L] = a_j[L] - y_j$$

Si la activación es sigmoide, su derivada respecto a la suma ponderada es:

$$g'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

Y, como la suma ponderada es lineal en los parámetros, las derivadas respecto al peso y al sesgo son inmediatas:

$$\partial z_j[L] / \partial w_{ij}[L] = a_i[L-1]$$

$$\partial z_j[L] / \partial b_j[L] = 1$$

Agrupando las dos primeras derivadas aparece una cantidad fundamental en backpropagation, el error local o delta de la neurona de salida:

$$\delta_j[L] = (\partial C / \partial a_j[L]) \cdot (\partial a_j[L] / \partial z_j[L])$$

$$\delta_j[L] = (a_j[L] - y_j) \cdot g'(z_j[L])$$

Este término puede interpretarse como la responsabilidad de la neurona en el error final: cuanto mayor sea  $\delta$ , mayor será la corrección que habrá que aplicar a sus parámetros.

#### 12.2.4. Gradientes finales para pesos y sesgos en la salida.

Una vez definido el término  $\delta$ , las expresiones de los gradientes se simplifican mucho:

$$\partial C / \partial b_j[L] = \delta_j[L]$$

$$\partial C / \partial w_{ij}[L] = a_i[L-1] \cdot \delta_j[L]$$

La interpretación es muy natural. El gradiente del sesgo coincide exactamente con el error local de la neurona. El gradiente del peso es ese error multiplicado por la activación que llega desde la capa anterior. Por tanto, un peso se corregirá mucho cuando la neurona de destino tenga un error importante y, además, la neurona de origen haya estado muy activa.

#### 12.2.5. Cómo se retropropaga el error a una capa oculta.

En una capa oculta, la neurona no influye directamente en el coste. Su efecto pasa primero por la capa siguiente. Por eso, el error local de una neurona  $j$  en una capa  $\ell$  se define propagando hacia atrás los deltas de la capa posterior:

$$\delta_j[\ell] = (\sum_k w_{jk}[\ell+1] \cdot \delta_k[\ell+1]) \cdot g'(z_j[\ell])$$

Esta expresión resume la idea central de la retropropagación: el error de la capa siguiente se reparte hacia atrás a través de los pesos que conectan ambas capas y, después, se ajusta según la derivada de la activación de la neurona actual.

Una vez conocido  $\delta_j[\ell]$ , los gradientes vuelven a adoptar la misma forma que en la última capa:

$$\partial C / \partial b_j[\ell] = \delta_j[\ell]$$

$$\partial C / \partial w_{ij}[\ell] = a_i[\ell-1] \cdot \delta_j[\ell]$$

#### 12.2.6. Forma matricial del algoritmo.

Si agrupamos todas las neuronas de una capa en vectores, el resultado se puede escribir de forma mucho más compacta:

$$\delta[L] = (a[L] - y) \odot g'(z[L])$$

$$\delta[\ell] = ((W[\ell+1])^T \cdot \delta[\ell+1]) \odot g'(z[\ell])$$

$$\partial C / \partial W[\ell] = \delta[\ell] \cdot (a[\ell-1])^T$$

$$\partial C / \partial b[\ell] = \delta[\ell]$$

Aquí  $\odot$  representa el producto elemento a elemento. Esta forma matricial es la que se emplea habitualmente en implementaciones reales, porque permite calcular todos los gradientes de una capa a la vez de manera muy eficiente.

#### 12.2.7. Resumen operativo y actualización por gradiente.

Visto como procedimiento, backpropagation sigue siempre la misma secuencia:

- Se calcula el error de la última capa.
- Ese error se retropropaga hacia las capas anteriores.
- A partir de los deltas se obtienen las derivadas de pesos y sesgos.

- Finalmente, los parámetros se actualizan mediante descenso por gradiente.

$$W[\ell] \leftarrow W[\ell] - \eta \cdot \partial C / \partial W[\ell]$$

$$b[\ell] \leftarrow b[\ell] - \eta \cdot \partial C / \partial b[\ell]$$

La idea central es sencilla aunque la notación pueda parecer densa al principio: primero calculamos cuánto influye la salida final en el error, después trasladamos esa información hacia atrás capa a capa y, por último, convertimos ese error local en derivadas respecto a pesos y sesgos. Gracias a ello, no hace falta volver a derivar toda la red desde cero para cada parámetro.

### 12.3. Relación entre activación y estabilidad del gradiente.

Las funciones de activación no solo afectan a la expresividad, sino también a la estabilidad numérica del entrenamiento. Activaciones saturantes como sigmoide o tanh pueden producir gradientes pequeños en regiones extremas. ReLU evita parte de ese efecto en la zona positiva, lo que ayudó históricamente al entrenamiento de redes profundas.

Sin embargo, la elección de activación debe considerarse junto a inicialización, normalización y arquitectura. La estabilidad es un fenómeno sistémico, no el resultado de una sola decisión aislada.

### 12.4. Regularización desde un punto de vista formal.

La regularización introduce restricciones que penalizan soluciones excesivamente complejas. En L2 se añade a la pérdida un término proporcional a la suma de los cuadrados de los pesos. En L1 se penaliza la suma de los valores absolutos. Estas técnicas reducen la tendencia a ajustes extremos y favorecen soluciones más suaves.

Dropout puede interpretarse como una regularización estocástica. Durante el entrenamiento obliga a la red a no depender demasiado de rutas concretas de activación, lo que favorece representaciones más robustas.

### 12.5. Sesgo-varianza y elección del modelo.

Una forma clásica de interpretar el rendimiento de un modelo es mediante el compromiso entre sesgo y varianza. Un modelo demasiado simple tiene alto sesgo: no capta bien la estructura del problema. Un modelo demasiado flexible puede tener alta varianza: se adapta demasiado a las fluctuaciones del entrenamiento.

Las redes neuronales, por su alta capacidad, pueden moverse con facilidad hacia el extremo de la varianza si no se controlan adecuadamente. De ahí la importancia de validación, regularización y comparación con modelos más sencillos.

### 12.6. Transfer learning.

El aprendizaje por transferencia consiste en reutilizar un modelo preentrenado en una tarea relacionada. En vez de aprender todos los patrones desde cero, aprovechamos representaciones que el modelo ya ha adquirido en un conjunto grande de datos. Esto es extremadamente útil cuando nuestro dataset es limitado.

En visión, por ejemplo, se puede tomar una CNN preentrenada y reajustar solo las últimas capas. En lenguaje, se usan embeddings o transformers ya entrenados sobre corpora masivos. Esta estrategia reduce tiempo de entrenamiento y mejora el rendimiento en muchos escenarios reales.